



Nombre del estudiante: Jandony Guzman Garofalo

Paralelo: 1

REPORTE #6 GESTIÓN DE ENERGÍA EN EL ESP32

1. Descripción del ejercicio

En este trabajo implementé un sistema que ejecuta una tarea durante cinco segundos y luego entra en modo *deep sleep*. El ESP32 puede despertar de dos formas: automáticamente mediante el temporizador RTC o al presionar un pulsador conectado al GPIO33. El estado del sistema se muestra con un LED RGB y también se escribe en el monitor serial.

Escogí *deep sleep* porque en este modo la CPU y la mayoría de los periféricos digitales dejan de funcionar. Solo se mantienen los elementos del dominio RTC que se necesitan para conservar los contadores y detectar las fuentes de despertar.

1.1. Objetivos de la implementación

- Ejecutar una tarea activa durante un tiempo definido.
- Configurar el despertar por temporizador con `esp_sleep_enable_timer_wakeup()`.
- Configurar el despertar externo con `esp_sleep_enable_ext0_wakeup()`.
- Indicar los estados mediante un LED RGB y mensajes seriales.
- Organizar el código en funciones y archivos separados.

2. Circuito en Wokwi

El circuito utiliza un ESP32 DevKit v1, un LED RGB de cátodo común, tres resistencias de 220 Ω , un pulsador y una resistencia de 10 k Ω para mantener el GPIO33 en nivel alto mientras el botón no está presionado.

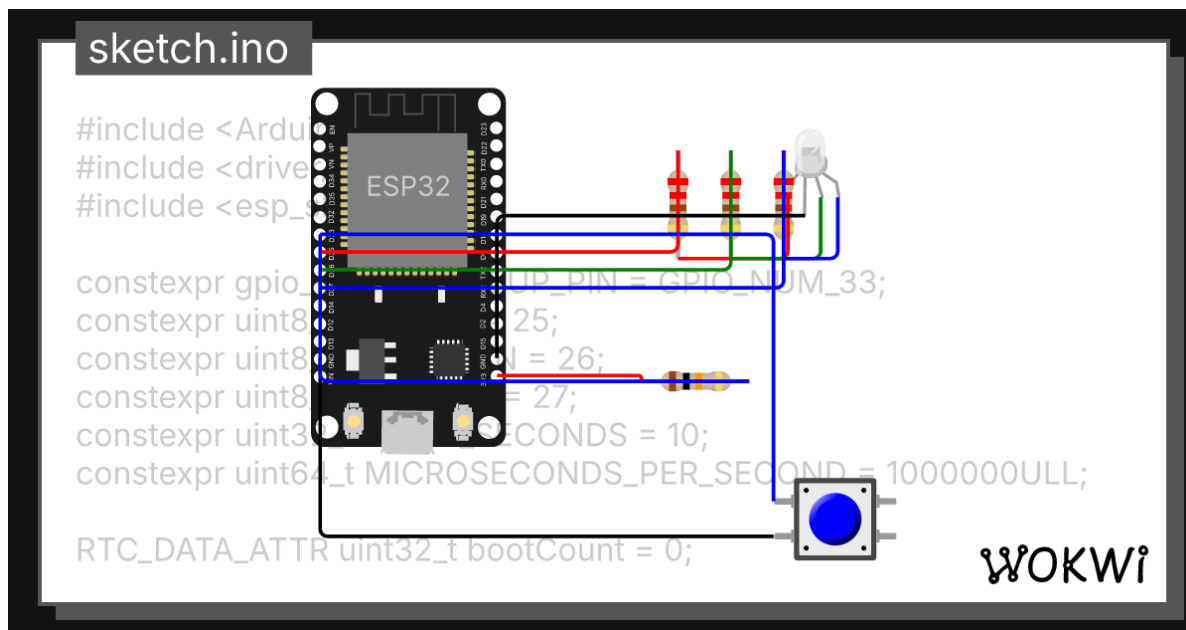


Figura 1: Circuito implementado y publicado en Wokwi.

Elemento	Conexión	Uso
Canal rojo	GPIO25, resistencia de 220 Ω y pin R	Preparación para dormir
Canal verde	GPIO26, resistencia de 220 Ω y pin G	Tarea activa
Canal azul	GPIO27, resistencia de 220 Ω y pin B	Arranque o despertar
Pulsador	GPIO33 y GND	Despertar externo EXT0
Resistencia pull-up	3.3 V, 10 k Ω y GPIO33	Mantener el pin en HIGH

Cuadro 1: Conexiones principales del circuito.

El circuito se puede abrir desde el siguiente vínculo: **proyecto en Wokwi**. Durante el reposo se puede hacer clic en el pulsador azul o mantener presionada la tecla B.

3. Funcionamiento del sistema

La secuencia se repite después de cada despertar:

1. El ESP32 inicia, lee la causa del arranque y enciende el LED azul.
2. Ejecuta la tarea activa durante cinco segundos. El LED verde parpadea una vez por segundo.
3. Enciende los canales rojo y verde al mismo tiempo para formar amarillo. En ese momento configura TIMER y EXT0.
4. Apaga el LED y entra en *deep sleep*.
5. Despierta al cumplirse diez segundos o cuando GPIO33 pasa a nivel bajo.

Estado	LED	Mensaje principal
Despierto	Azul	ESTADO=DESPIERTO
Tarea activa	Verde intermitente	ESTADO=ACTIVO
Preparando reposo	Amarillo	ESTADO=PREPARANDO_SLEEP
Deep sleep	Apagado	ESTADO=SLEEP

Cuadro 2: Estados indicados por el LED y el monitor serial.

3.1. Memoria RTC

El despertar desde *deep sleep* reinicia la ejecución desde `setup()`. Para comprobar que existió un ciclo anterior, declaré tres contadores con `RTC_DATA_ATTR`. Estos datos se mantienen mientras el dominio RTC siga alimentado.

```
RTC_DATA_ATTR uint32_t bootCount = 0;
RTC_DATA_ATTR uint32_t timerWakeCount = 0;
RTC_DATA_ATTR uint32_t externalWakeCount = 0;
```

4. Desarrollo del código

El proyecto principal se desarrolló en PlatformIO con el framework Arduino. Separé el manejo del LED y la configuración de energía para que el archivo principal solo contenga la secuencia del ejercicio.

Archivo	Contenido
src/main.cpp	Flujo principal, tarea activa y contadores RTC.
src/power_manager.cpp	Fuentes de despertar y entrada a deep sleep.
src/status_led.cpp	Control de los canales del LED RGB.
diagram.json	Componentes y conexiones del circuito Wokwi.
wokwi.toml	Rutas del firmware generado por PlatformIO.
tests/	Escenarios TIMER, EXT0 y validación del proyecto.

Cuadro 3: Organización del proyecto.

La configuración de las fuentes de despertar se realiza antes de apagar el LED y entrar en reposo:

```
esp_sleep_enable_timer_wakeup(
    SLEEP_SECONDS * MICROSECONDS_PER_SECOND);

esp_sleep_enable_ext0_wakeup(
    GPIO_NUM_33, LOW);

rtc_gpio_pullup_en(GPIO_NUM_33);
rtc_gpiopulldown_dis(GPIO_NUM_33);

esp_deep_sleep_start();
```

Antes de ejecutar `esp_deep_sleep_start()`, el programa espera que el botón esté liberado. Esto evita que el ESP32 se duerma con la condición EXT0 ya activa y se despierte de inmediato.

5. Pruebas realizadas

Primero compilé el proyecto completo con PlatformIO. También compilé por separado el archivo usado en la versión web de Wokwi para comprobar que ambas variantes sean válidas. Finalmente, revisé el archivo `diagram.json` con el linter de Wokwi y ejecuté un script que compara los pines del circuito con los pines usados en el código.

Prueba	Resultado	Observación
Compilación del proyecto PlatformIO	Aprobada	RAM: 6.9 %; flash: 21.5 %
Compilación de la variante web	Aprobada	Se generó el firmware BIN
Validación de <code>diagram.json</code>	Aprobada	Archivo JSON válido
Linter Wokwi CLI 0.26.1	Aprobada	Cero errores y una nota informativa
Comprobaciones de coherencia	Aprobada	20 de 20 verificaciones
Enlace Wokwi	Aprobada	Respuesta HTTP 200
Repositorio GitHub	Aprobada	Respuesta HTTP 200

Cuadro 4: Resumen de las pruebas ejecutadas.

5.1. Resultado de compilación

```
RAM:  [ =      ] 6.9% (22712 bytes de 327680 bytes)
Flash: [ ==    ] 21.5% (281561 bytes de 1310720 bytes)
===== [SUCCESS] =====
```

5.2. Prueba por temporizador

Para esta prueba se inicia la simulación y no se presiona el pulsador. Luego del mensaje `ESTADO=SLEEP`, el temporizador RTC debe despertar al ESP32 después de diez segundos. En el siguiente arranque la salida esperada es:

```
BOOT=2
CAUSA=temporizador RTC
CONTADORES timer=1, ext0=0
```

5.3. Prueba por pulsador EXT0

En esta prueba se espera hasta que el LED esté apagado y se presiona el botón `WAKE_EXT0`. El nuevo arranque debe identificar el GPIO33 como causa de despertar:

```
BOOT=2
CAUSA=pin externo EXT0 (GPIO33)
CONTADORES timer=0, ext0=1
```

Los escenarios automatizados se llaman `timer-wakeup.yaml` y `ext0-wakeup.yaml`. Su ejecución mediante Wokwi CI necesita un token personal, por lo que no se guardó ninguna credencial en el repositorio. Las compilaciones, el linter y las comprobaciones locales sí fueron ejecutadas en esta entrega.

6. Análisis del comportamiento

El programa permanece activo aproximadamente siete segundos: un segundo para indicar el arranque, cinco segundos para la tarea y un segundo para preparar el reposo. Después entra en *deep sleep* durante un

máximo de diez segundos. Este tiempo es corto porque facilita la demostración. En una aplicación alimentada con batería debería aumentarse según la frecuencia real con la que se necesiten datos.

Wokwi permite comprobar la lógica, los pines y la secuencia de mensajes, pero no reemplaza una medición de corriente. En una placa real también consumen energía el regulador, el conversor USB–serial y el LED integrado. Por esta razón no se incluyeron valores de corriente simulados ni se presentaron como si fueran mediciones físicas.

No se realizó una prueba con hardware real en esta entrega. Para completarla se debe alimentar la placa con una tensión conocida, medir la corriente activa y la corriente durante *deep sleep*, y documentar el instrumento utilizado.

7. Conclusiones

1. El modo *deep sleep* es apropiado cuando la aplicación puede permanecer inactiva durante periodos largos y reiniciar su ejecución después de cada despertar.
2. El temporizador RTC y EXT0 pueden habilitarse al mismo tiempo. El primer evento que ocurra despierta al microcontrolador y queda registrado como la causa del nuevo arranque.
3. La memoria RTC permite conservar contadores pequeños sin mantener encendida la memoria principal. Esto facilitó verificar los ciclos de reposo y despertar.

8. Recomendaciones

1. Ajustar el tiempo de reposo a la necesidad real del sistema y apagar también los sensores o cargas externas que no se utilicen.
2. Medir el consumo en una placa real antes de estimar la autonomía de una batería, debido a que los componentes adicionales del DevKit también consumen corriente.

9. Vínculos del proyecto

- Abrir circuito y simulación en Wokwi.
- Abrir repositorio con el código fuente.

Referencias

1. Espressif Systems, *ESP-IDF Programming Guide: Sleep Modes*.
2. Espressif Systems, *Arduino ESP32: Deep Sleep*.
3. Wokwi, *ESP32 Simulation*.
4. Wokwi, *Configuring Your Project*.